

```

In [1]: # Setup and Imports
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    confusion_matrix,
    roc_curve,
    auc,
    roc_auc_score,
    accuracy_score,
    ConfusionMatrixDisplay
)
import warnings
from IPython.display import display # For displaying DataFrames nicely

# Configuration
sns.set_style("whitegrid")
plt.rcParams["figure.figsize"] = (8, 6)
warnings.filterwarnings('ignore', category=UserWarning)

# Define random state for reproducibility
RANDOM_STATE = 42

print("Libraries imported and configuration set.")

```

Libraries imported and configuration set.

```

In [2]: # Load dataset
df = pd.read_csv("wc1.csv")

# Convert the categorical 'type' column to binary (0: alexa, 1: phish)
df['type'] = df['type'].map({'alexa': 0, 'phish': 1})

# Define Features (X) and Target (y)
X = df.drop('type', axis=1)
y = df['type']

# Split data into training and testing sets (67/33 split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=RANDOM_STATE, stratify=y
)

print("Data loaded and split into training and testing sets.")
print(f"Training set size: {X_train.shape[0]} samples")
print(f"Test set size: {X_test.shape[0]} samples")

```

Data loaded and split into training and testing sets.

Training set size: 16235 samples

Test set size: 7997 samples

```
In [3]: # Initialize Classification Algorithms

# Logistic Regression (LR)
lr_model = LogisticRegression(random_state=RANDOM_STATE, solver='liblinear')

# Random Forest (RF)
rf_model = RandomForestClassifier(n_estimators=300, random_state=RANDOM_STATE, n_jo

# Neural Network (NN) / Multi-Layer Perceptron (MLP)
mlp_model = MLPClassifier(hidden_layer_sizes=(100, 50),
                           max_iter=200,
                           random_state=RANDOM_STATE,
                           solver='adam')

print("Models initialized.")
```

Models initialized.

```
In [4]: # Train all models on the UN-SCALED training data
print("Training models on unscaled data...")
lr_model.fit(X_train, y_train)
rf_model.fit(X_train, y_train)
mlp_model.fit(X_train, y_train)
print("Training complete.")

# Store predictions and probabilities for ROC/CM analysis
lr_pred = lr_model.predict(X_test)
lr_prob = lr_model.predict_proba(X_test)[:, 1]
rf_pred = rf_model.predict(X_test)
rf_prob = rf_model.predict_proba(X_test)[:, 1]
mlp_pred = mlp_model.predict(X_test)
mlp_prob = mlp_model.predict_proba(X_test)[:, 1]

# Calculate Confusion Matrices
cm_lr = confusion_matrix(y_test, lr_pred)
cm_rf = confusion_matrix(y_test, rf_pred)
cm_mlp = confusion_matrix(y_test, mlp_pred)

# Store AUC scores
lr_auc = roc_auc_score(y_test, lr_prob)
rf_auc = roc_auc_score(y_test, rf_prob)
mlp_auc = roc_auc_score(y_test, mlp_prob)

# Initialize AUC dictionary for final summary
auc_results = {
    'LR (Unscaled)': lr_auc,
    'RF (Unscaled)': rf_auc,
    'NN (Unscaled)': mlp_auc
}
```

Training models on unscaled data...

Training complete.

```
In [5]: # --- Confusion Matrices (Unscaled Data) ---

print("--- Confusion Matrices (Unscaled Data) ---")
```

```

# 1. Logistic Regression
print("\nLogistic Regression Confusion Matrix:")
print(cm_lr)
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_lr, display_labels=['Alexa (0)', 'Phish'])
plt.title('LR Confusion Matrix (Unscaled Data)')
plt.show()

# 2. Random Forest
print("\nRandom Forest Confusion Matrix:")
print(cm_rf)
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_rf, display_labels=['Alexa (0)', 'Phish'])
plt.title('RF Confusion Matrix (Unscaled Data)')
plt.show()

# 3. Neural Network
print("\nNeural Network Confusion Matrix:")
print(cm_mlp)
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_mlp, display_labels=['Alexa (0)', 'Phish'])
plt.title('NN Confusion Matrix (Unscaled Data)')
plt.show()

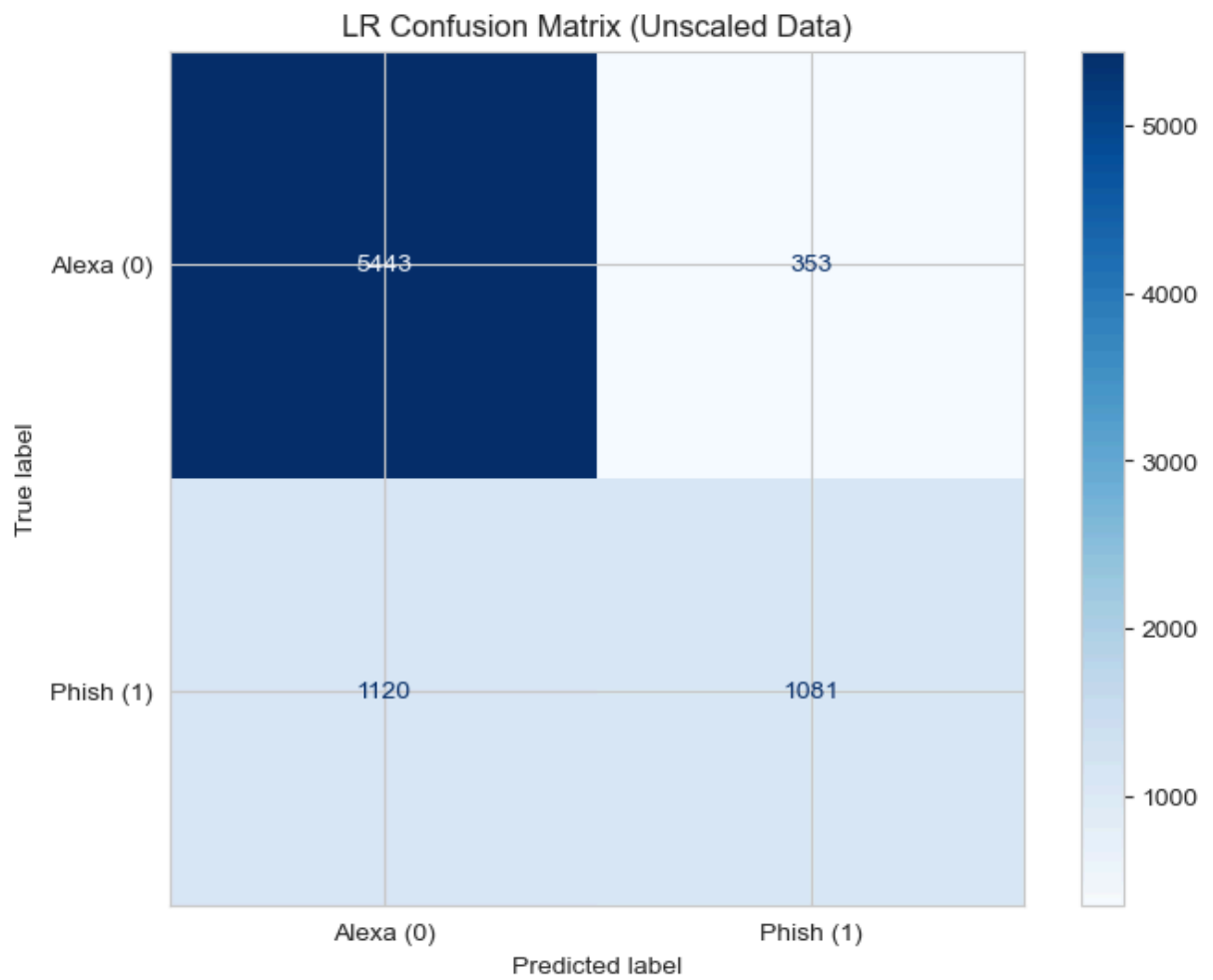
```

--- Confusion Matrices (Unscaled Data) ---

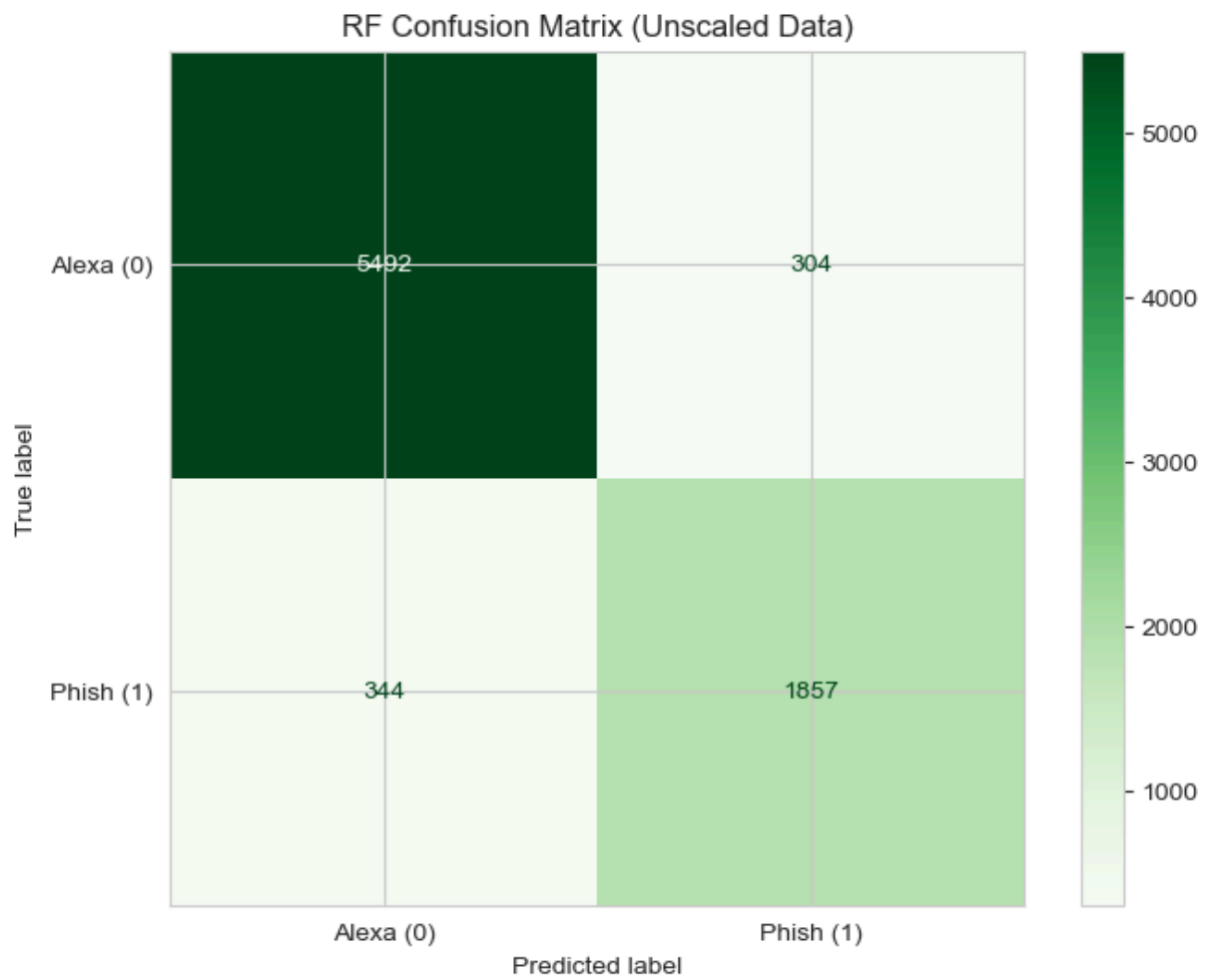
```

Logistic Regression Confusion Matrix:
[[5443  353]
 [1120 1081]]
<Figure size 500x500 with 0 Axes>

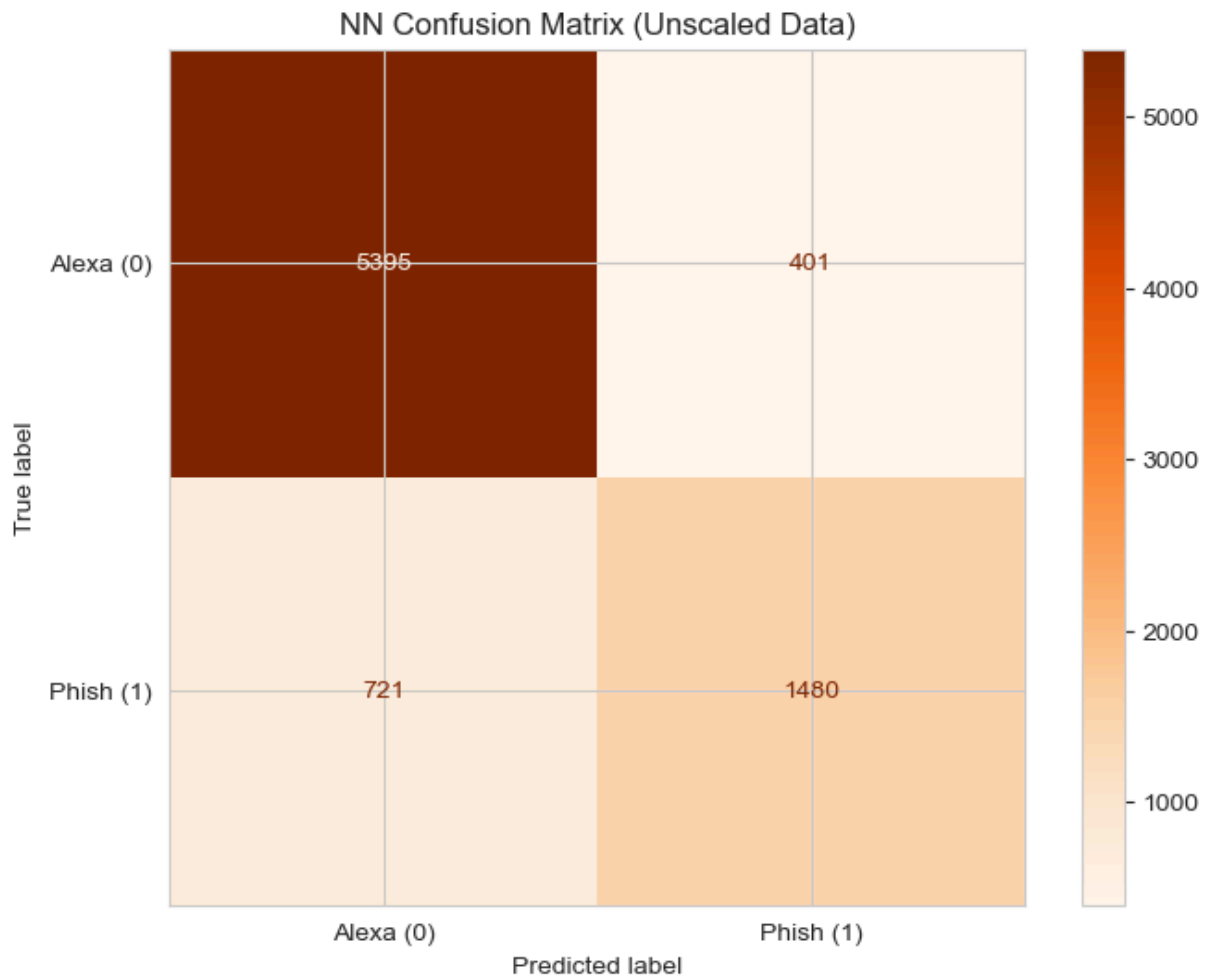
```



Random Forest Confusion Matrix:
[[5492 304]
 [344 1857]]
<Figure size 500x500 with 0 Axes>



Neural Network Confusion Matrix:
[[5395 401]
 [721 1480]]
<Figure size 500x500 with 0 Axes>



```
In [6]: # 1. ROC curves for each of the classification algorithms (LR, RF, and NN)

# Define the models and their probabilities/AUC scores for iteration
models_to_plot = [
    ('Logistic Regression (LR) Unscaled', lr_prob, lr_auc),
    ('Random Forest (RF) Unscaled', rf_prob, rf_auc),
    ('Neural Network (NN) Unscaled', mlp_prob, mlp_auc)
]

print("--- Q1: ROC AUC Scores (Unscaled Data) ---")

# Iterate and plot each ROC curve separately
for name, y_prob, roc_auc in models_to_plot:

    # Calculate False Positive Rate and True Positive Rate
    fpr, tpr, _ = roc_curve(y_test, y_prob)

    plt.figure(figsize=(8, 6))

    # Plot the ROC curve for the current model
    plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:0.2f})')

    # Plot the baseline (random classifier)
    plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Chance (AUC = 0.5)')
```

```

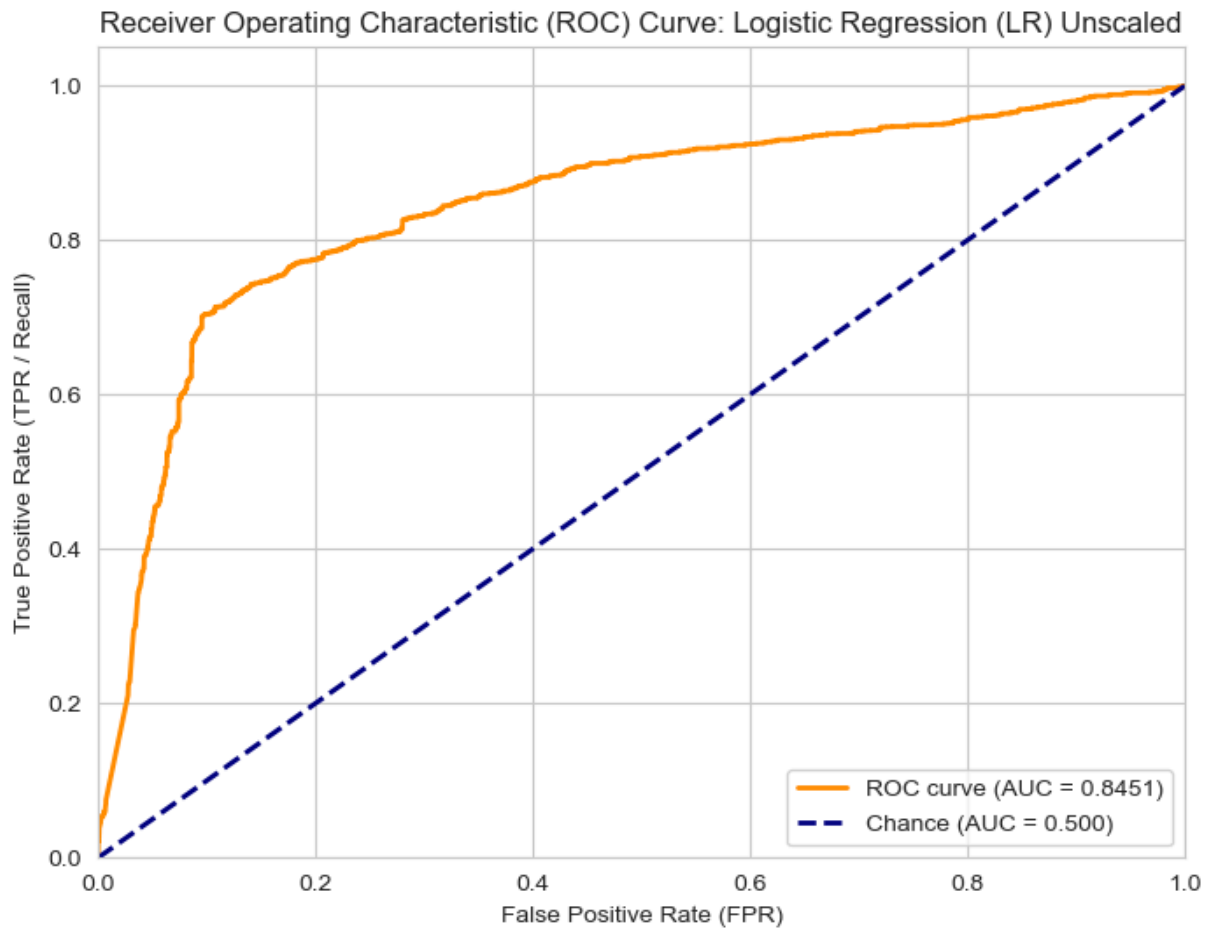
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate (FPR)')
plt.ylabel('True Positive Rate (TPR / Recall)')
plt.title(f'Receiver Operating Characteristic (ROC) Curve: {name}')
plt.legend(loc="lower right")
plt.grid(True)
plt.show()

print(f"{name}: {roc_auc:.4f}")

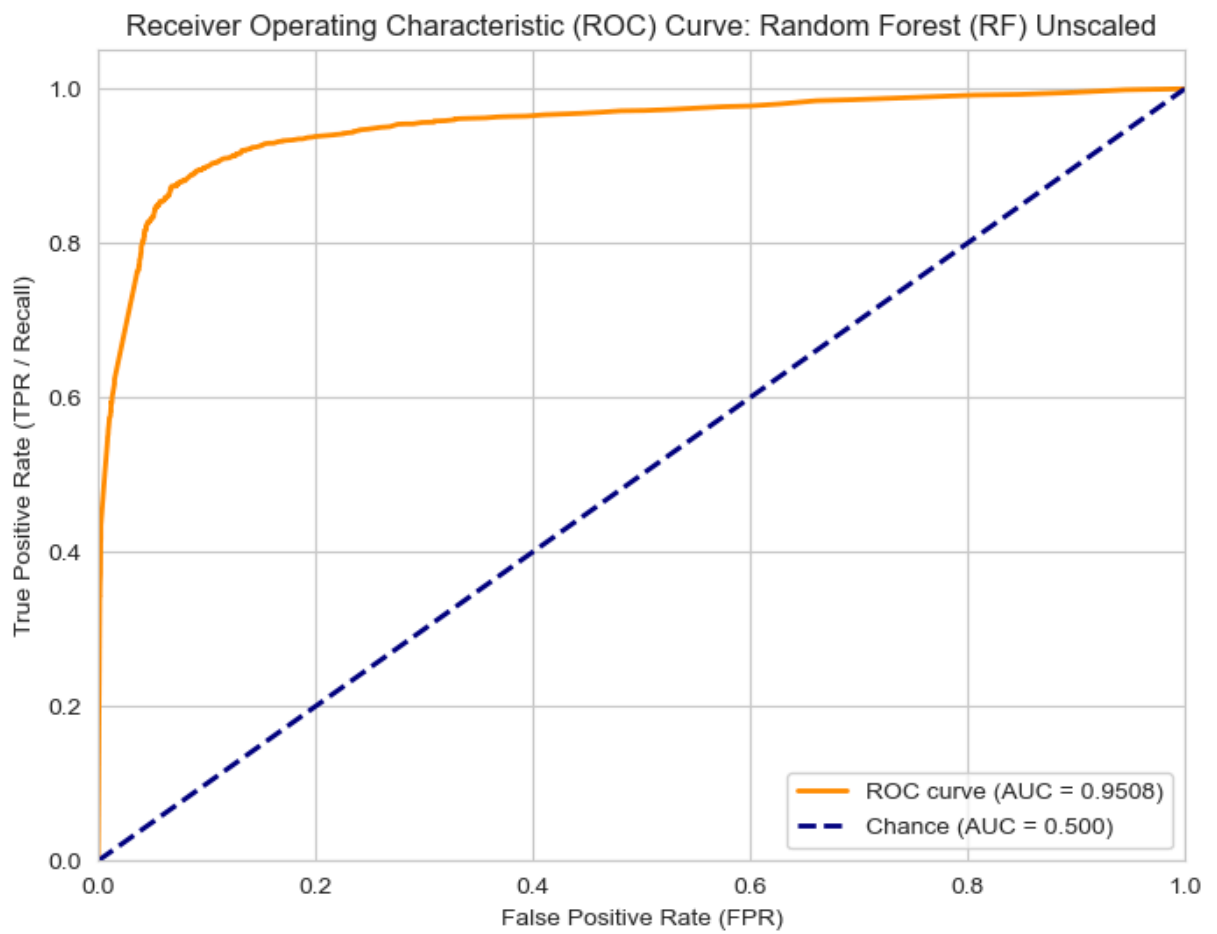
best_roc_model = max(auc_results, key=auc_results.get)
print(f"\nConclusion for Q1 (Best ROC/AUC): The **{best_roc_model.split(' ')[0]}**

```

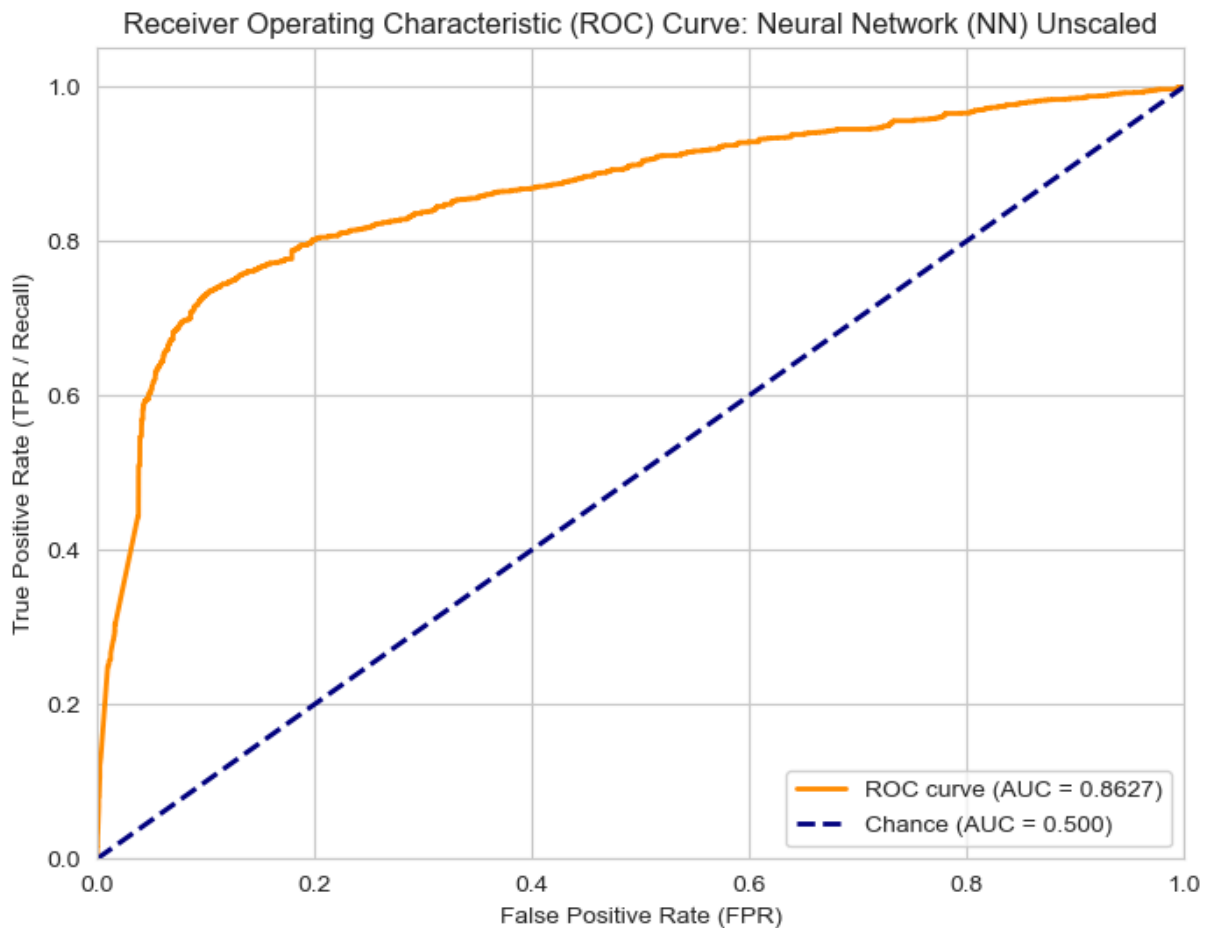
--- Q1: ROC AUC Scores (Unscaled Data) ---



Logistic Regression (LR) Unscaled: 0.8451



Random Forest (RF) Unscaled: 0.9508



Neural Network (NN) Unscaled: 0.8627

Conclusion for Q1 (Best ROC/AUC): The **RF** algorithm gives the best result for this problem with an AUC of **0.9508**.

```
In [7]: # 2. Apply 10-fold cross validation to each of the classification algorithms (LR, R

# Use the full unscaled dataset (X, y) for cross-validation
k_folds = KFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

print("Starting 10-Fold Cross-Validation on UN-SCALED Data (Accuracy):")

# Logistic Regression CV
lr_cv_scores = cross_val_score(lr_model, X, y, cv=k_folds, scoring='accuracy', n_jo
lr_cv_mean = np.mean(lr_cv_scores)
print(f"LR CV Mean Accuracy: {lr_cv_mean:.4f} (Std Dev: {np.std(lr_cv_scores):.4f})

# Random Forest CV
rf_cv_scores = cross_val_score(rf_model, X, y, cv=k_folds, scoring='accuracy', n_jo
rf_cv_mean = np.mean(rf_cv_scores)
print(f"RF CV Mean Accuracy: {rf_cv_mean:.4f} (Std Dev: {np.std(rf_cv_scores):.4f})

# Neural Network CV
mlp_cv_scores = cross_val_score(mlp_model, X, y, cv=k_folds, scoring='accuracy', n_
mlp_cv_mean = np.mean(mlp_cv_scores)
print(f"NN CV Mean Accuracy: {mlp_cv_mean:.4f} (Std Dev: {np.std(mlp_cv_scores):.4f}

# Initialize CV dictionary for final summary
```

```

cv_results = {
    'LR (Unscaled)': lr_cv_mean,
    'RF (Unscaled)': rf_cv_mean,
    'NN (Unscaled)': mlp_cv_mean
}

best_cv_model = max(cv_results, key=cv_results.get)
print(f"\nConclusion for Q2 (Best CV): The **{best_cv_model.split(' ')[0]** algorithm

```

Starting 10-Fold Cross-Validation on UN-SCALED Data (Accuracy):

LR CV Mean Accuracy: 0.8126 (Std Dev: 0.0186)

RF CV Mean Accuracy: 0.9273 (Std Dev: 0.0054)

NN CV Mean Accuracy: 0.8391 (Std Dev: 0.0317)

Conclusion for Q2 (Best CV): The **RF** algorithm leads to the best performance with 10-fold cross-validation on unscaled data with an accuracy of **0.9273**.

In [8]: *# 3. Normalize the attributes*

```

# Initialize StandardScaler
scaler = StandardScaler()

# Fit the scaler on the training data and transform both training and test data
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert back to DataFrame
X_train_scaled = pd.DataFrame(X_train_scaled, columns=X_train.columns)
X_test_scaled = pd.DataFrame(X_test_scaled, columns=X_test.columns)

print("Data successfully normalized (StandardScaler applied).")

```

Data successfully normalized (StandardScaler applied).

In [9]: *# Re-evaluate Logistic Regression on Normalized Data*

```

# Re-initialize the model
lr_model_n = LogisticRegression(random_state=RANDOM_STATE, solver='lbfgs')

# Train model
lr_model_n.fit(X_train_scaled, y_train)

# Predict and evaluate
lr_pred_n = lr_model_n.predict(X_test_scaled)
lr_prob_n = lr_model_n.predict_proba(X_test_scaled)[: , 1]
lr_auc_n = roc_auc_score(y_test, lr_prob_n)
lr_acc_n = accuracy_score(y_test, lr_pred_n)
cm_lr_n = confusion_matrix(y_test, lr_pred_n)

# Update AUC results
auc_results['LR (Scaled)'] = lr_auc_n

print(f"--- Logistic Regression (Normalized) ---")
print(f"Test Accuracy: {lr_acc_n:.4f}")
print(f"ROC AUC Score: {lr_auc_n:.4f}")
print("Confusion Matrix:\n", cm_lr_n)

```

```

# Plot Confusion Matrix
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_lr_n, display_labels=['Alexa (0)', 'Phish (1)'])
plt.title('LR Confusion Matrix (Normalized Data)')
plt.show()

# Plot ROC curve
plt.figure(figsize=(6, 6))
lr_fpr_n, lr_tpr_n, _ = roc_curve(y_test, lr_prob_n)
plt.plot(lr_fpr_n, lr_tpr_n, color='red', lw=2, label=f'ROC curve (AUC = {lr_auc_n:0.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.title('Logistic Regression ROC Curve (Normalized)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()

```

--- Logistic Regression (Normalized) ---

Test Accuracy: 0.8176

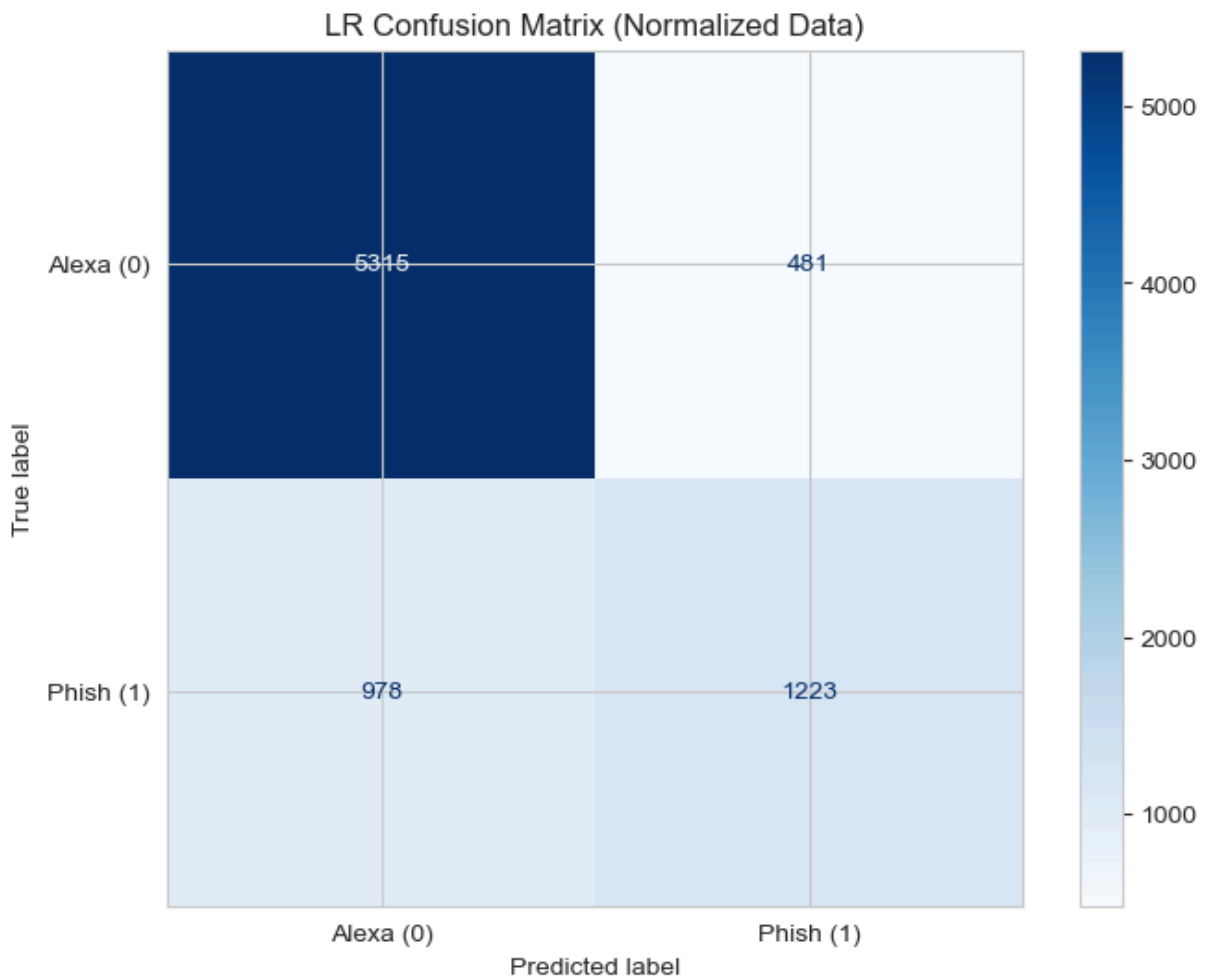
ROC AUC Score: 0.8420

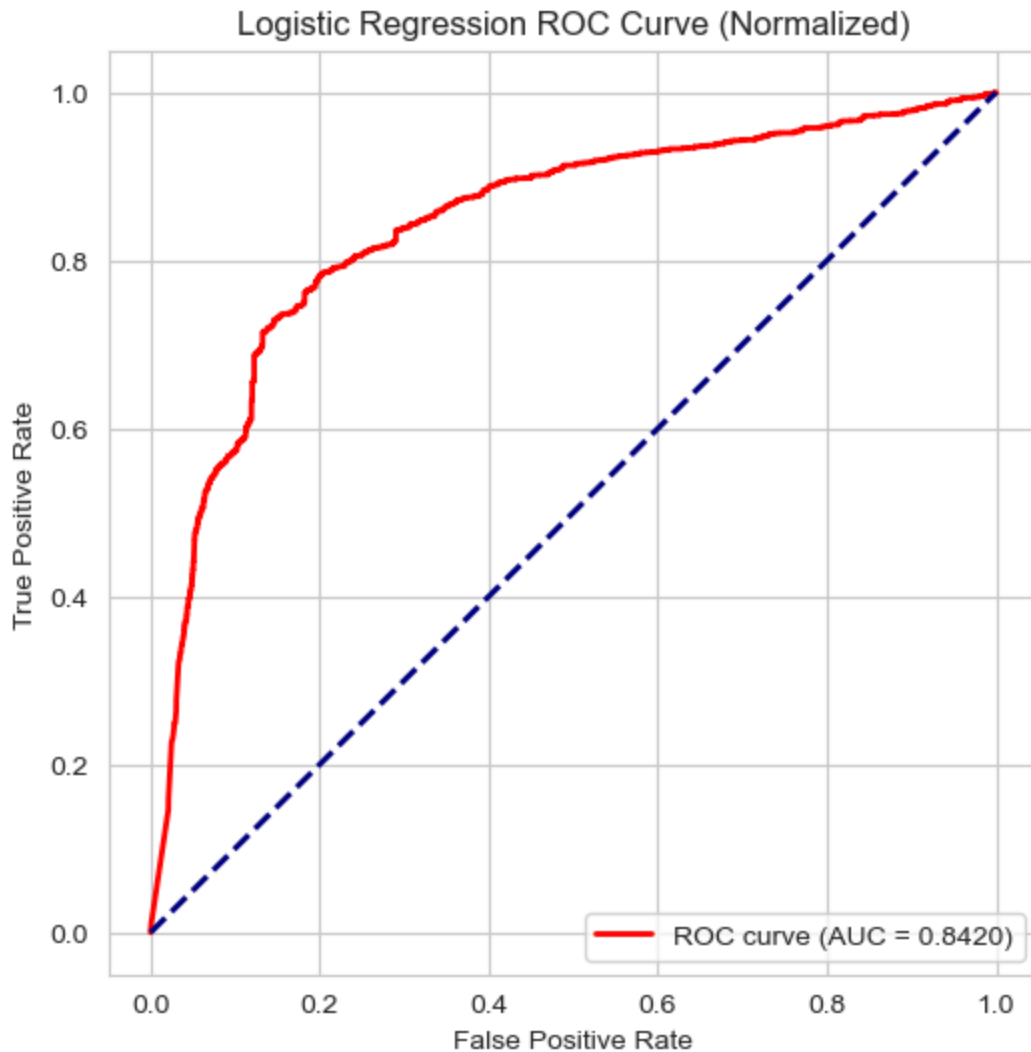
Confusion Matrix:

```
[[5315  481]
```

```
[ 978 1223]]
```

<Figure size 500x500 with 0 Axes>





```
In [10]: # Re-evaluate Random Forest on Normalized Data

# Re-initialize the model
rf_model_n = RandomForestClassifier(n_estimators=300, random_state=RANDOM_STATE, n_

# Train model
rf_model_n.fit(X_train_scaled, y_train)

# Predict and evaluate
rf_pred_n = rf_model_n.predict(X_test_scaled)
rf_prob_n = rf_model_n.predict_proba(X_test_scaled)[: , 1]
rf_auc_n = roc_auc_score(y_test, rf_prob_n)
rf_acc_n = accuracy_score(y_test, rf_pred_n)
cm_rf_n = confusion_matrix(y_test, rf_pred_n)

# Update AUC results
auc_results['RF (Scaled)'] = rf_auc_n

print(f"--- Random Forest (Normalized) ---")
print(f"Test Accuracy: {rf_acc_n:.4f}")
print(f"ROC AUC Score: {rf_auc_n:.4f}")
print("Confusion Matrix:\n", cm_rf_n)
```

```

# Plot Confusion Matrix
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_rf_n, display_labels=['Alexa (0)', 'Phish (1)'])
plt.title('RF Confusion Matrix (Normalized Data)')
plt.show()

# Plot ROC curve
plt.figure(figsize=(6, 6))
rf_fpr_n, rf_tpr_n, _ = roc_curve(y_test, rf_prob_n)
plt.plot(rf_fpr_n, rf_tpr_n, color='green', lw=2, label=f'ROC curve (AUC = {rf_auc_n:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.title('Random Forest ROC Curve (Normalized)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()

```

--- Random Forest (Normalized) ---

Test Accuracy: 0.9191

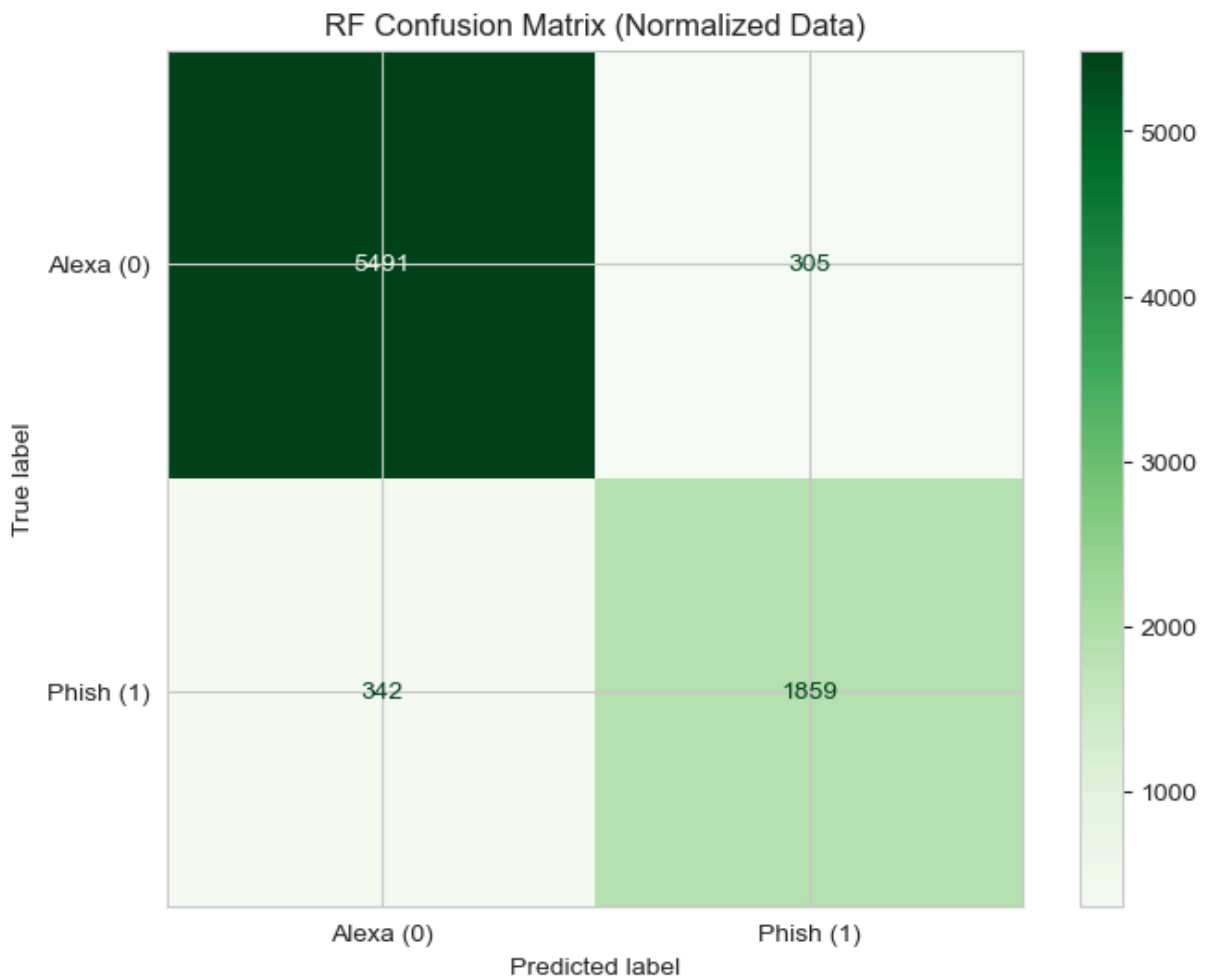
ROC AUC Score: 0.9507

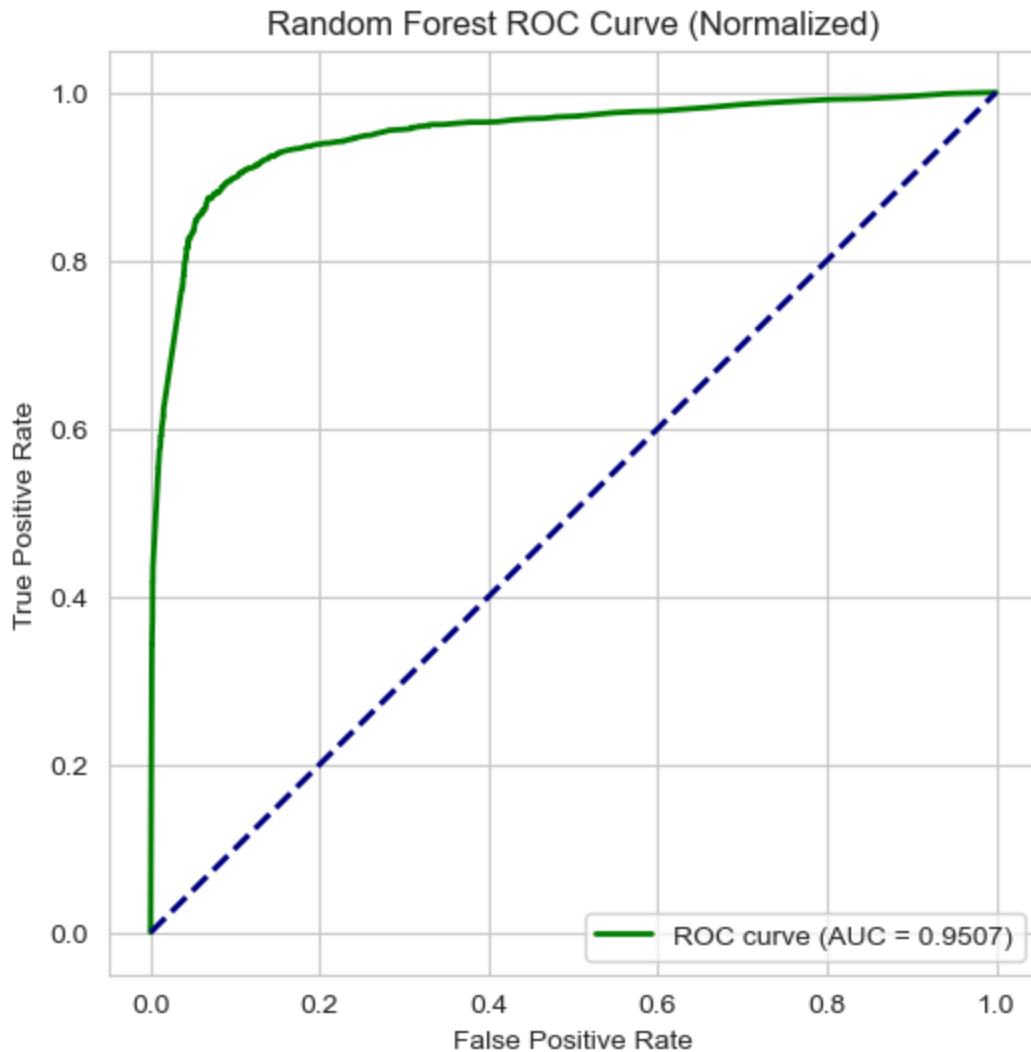
Confusion Matrix:

```
[[5491  305]
```

```
 [ 342 1859]]
```

<Figure size 500x500 with 0 Axes>





```
In [11]: # Re-evaluate Neural Network on Normalized Data

# Re-initialize the model
mlp_model_n = MLPClassifier(hidden_layer_sizes=(100, 50),
                             max_iter=200,
                             random_state=RANDOM_STATE,
                             solver='adam')

# Train model
mlp_model_n.fit(X_train_scaled, y_train)

# Predict and evaluate
mlp_pred_n = mlp_model_n.predict(X_test_scaled)
mlp_prob_n = mlp_model_n.predict_proba(X_test_scaled)[:, 1]
mlp_auc_n = roc_auc_score(y_test, mlp_prob_n)
mlp_acc_n = accuracy_score(y_test, mlp_pred_n)
cm_mlp_n = confusion_matrix(y_test, mlp_pred_n)

# Update AUC results
auc_results['NN (Scaled)'] = mlp_auc_n

print(f"--- Neural Network (Normalized) ---")
print(f"Test Accuracy: {mlp_acc_n:.4f}")
```

```

print(f"ROC AUC Score: {mlp_auc_n:.4f}")
print("Confusion Matrix:\n", cm_mlp_n)

# Plot Confusion Matrix
plt.figure(figsize=(5, 5))
ConfusionMatrixDisplay(confusion_matrix=cm_mlp_n, display_labels=['Alexa (0)', 'Phi
plt.title('NN Confusion Matrix (Normalized Data)')
plt.show()

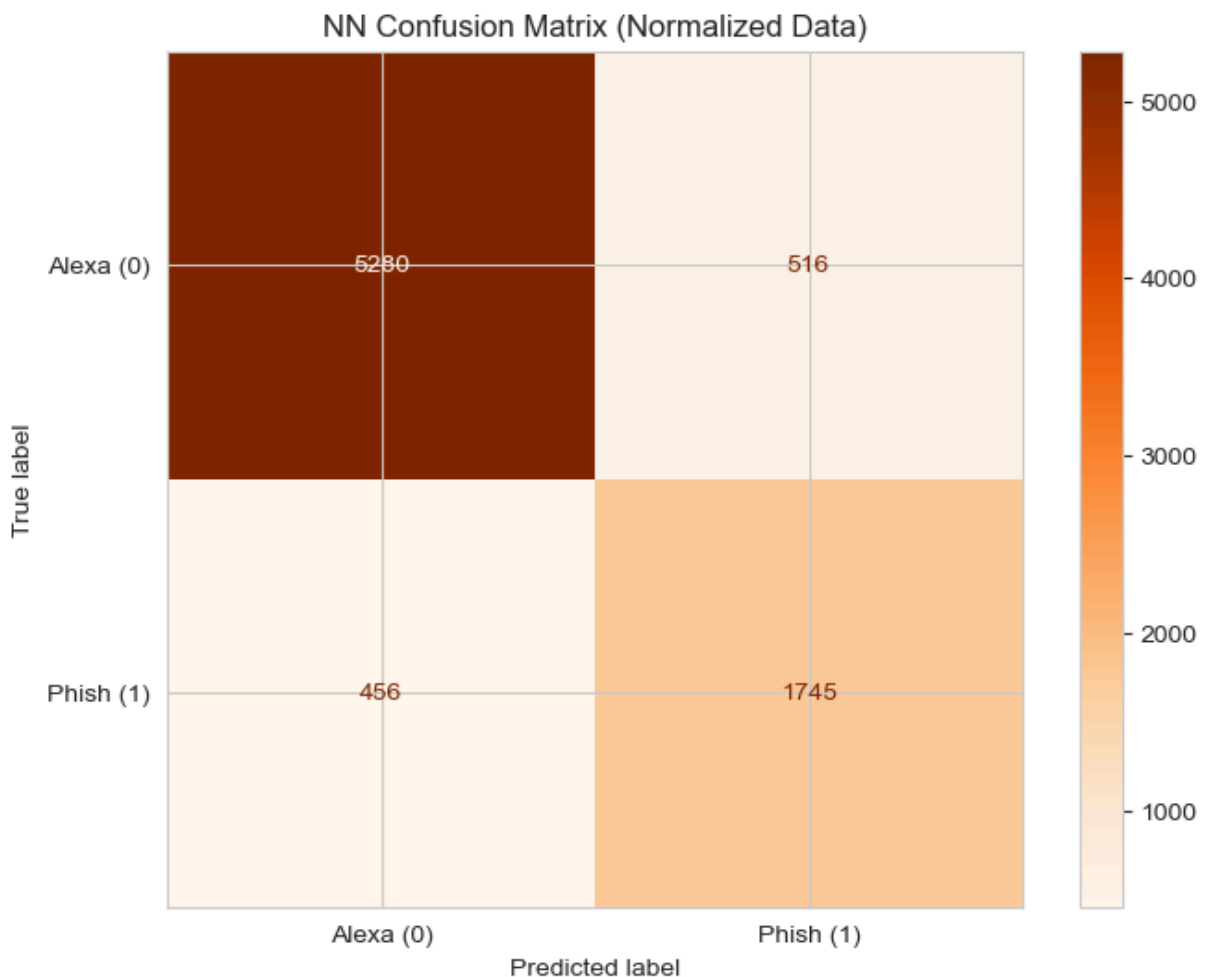
# Plot ROC curve
plt.figure(figsize=(6, 6))
mlp_fpr_n, mlp_tpr_n, _ = roc_curve(y_test, mlp_prob_n)
plt.plot(mlp_fpr_n, mlp_tpr_n, color='darkorange', lw=2, label=f'ROC curve (AUC = {
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.title('Neural Network ROC Curve (Normalized)')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend(loc="lower right")
plt.show()

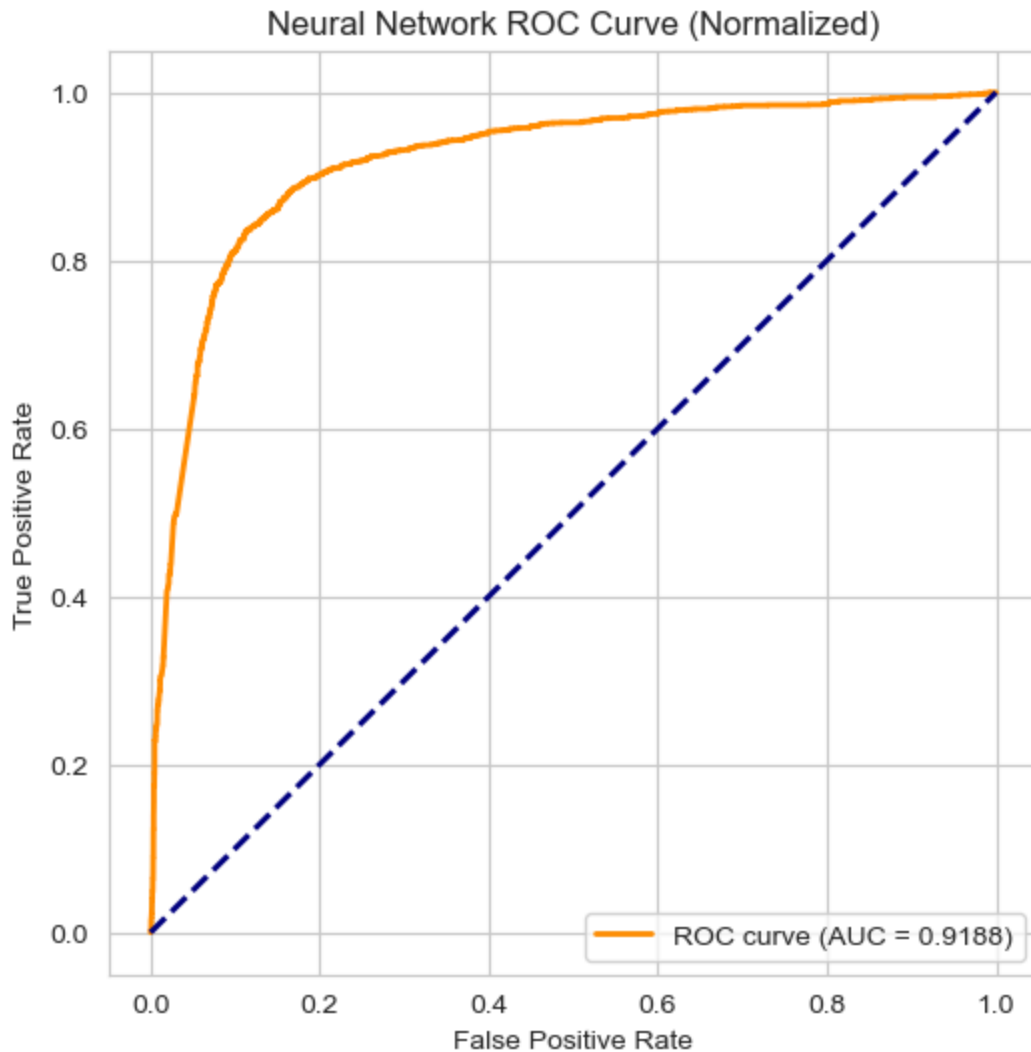
```

```

--- Neural Network (Normalized) ---
Test Accuracy: 0.8785
ROC AUC Score: 0.9188
Confusion Matrix:
[[5280  516]
 [ 456 1745]]
<Figure size 500x500 with 0 Axes>

```





```
In [12]: # Repeat 10-Fold Cross-Validation on the full NORMALIZED data

# Scale the full dataset X for cross-validation
X_scaled_full = scaler.fit_transform(X)

print("Starting 10-Fold Cross-Validation on NORMALIZED Data (Accuracy):")

# Logistic Regression CV (Normalized)
lr_cv_scores_n = cross_val_score(lr_model_n, X_scaled_full, y, cv=k_folds, scoring=
cv_results['LR (Scaled)'] = np.mean(lr_cv_scores_n)

# Random Forest CV (Normalized)
rf_cv_scores_n = cross_val_score(rf_model_n, X_scaled_full, y, cv=k_folds, scoring=
cv_results['RF (Scaled)'] = np.mean(rf_cv_scores_n)

# Neural Network CV (Normalized)
mlp_cv_scores_n = cross_val_score(mlp_model_n, X_scaled_full, y, cv=k_folds, scorin
cv_results['NN (Scaled)'] = np.mean(mlp_cv_scores_n)

print(f"LR CV Mean Accuracy (Normalized): {cv_results['LR (Scaled)']:.4f}")
print(f"RF CV Mean Accuracy (Normalized): {cv_results['RF (Scaled)']:.4f}")
print(f"NN CV Mean Accuracy (Normalized): {cv_results['NN (Scaled)']:.4f}")
```


Starting 10-Fold Cross-Validation on NORMALIZED Data (Accuracy):

LR CV Mean Accuracy (Normalized): 0.8256

RF CV Mean Accuracy (Normalized): 0.9272

NN CV Mean Accuracy (Normalized): 0.8873

In [13]: *# 3. Compare and contrast results from all LR, RF, and NN*

```
# Consolidate results into a DataFrame for easy comparison
summary_df = pd.DataFrame({
    'Metric': ['ROC AUC Score', '10-Fold CV Accuracy'],
    'LR (Unscaled)': [auc_results['LR (Unscaled)'], cv_results['LR (Unscaled)']],
    'RF (Unscaled)': [auc_results['RF (Unscaled)'], cv_results['RF (Unscaled)']],
    'NN (Unscaled)': [auc_results['NN (Unscaled)'], cv_results['NN (Unscaled)']],
    'LR (Normalized)': [auc_results['LR (Scaled)'], cv_results['LR (Scaled)']],
    'RF (Normalized)': [auc_results['RF (Scaled)'], cv_results['RF (Scaled)']],
    'NN (Normalized)': [auc_results['NN (Scaled)'], cv_results['NN (Scaled)']]
}).set_index('Metric')

print("--- Overall Model Performance Summary ---")
display(summary_df.T.style.highlight_max(axis=0, props='font-weight: bold; background-color: #d9e1f2'))

print("\n--- Comparison and Contrast ---")

# Best Overall Model
best_roc = summary_df.loc['ROC AUC Score'].idxmax()
best_cv = summary_df.loc['10-Fold CV Accuracy'].idxmax()

print(f"\nBest Algorithm Overall (by AUC): **{best_roc}** with **{summary_df.loc['ROC AUC Score'].values[best_roc]}**")
print(f"Best Algorithm Overall (by CV Acc): **{best_cv}** with **{summary_df.loc['10-Fold CV Accuracy'].values[best_cv]}**")

print("\n### Impact of Normalization:")

# LR Comparison
lr_change_auc = auc_results['LR (Scaled)'] - auc_results['LR (Unscaled)']
lr_change_cv = cv_results['LR (Scaled)'] - cv_results['LR (Unscaled)']
print(f"1. **Logistic Regression (LR)**: Performance improved after normalization")
print(f"    - AUC Change: {lr_change_auc:+.4f}")
print(f"    - CV Acc Change: {lr_change_cv:+.4f}")

# RF Comparison
rf_change_auc = auc_results['RF (Scaled)'] - auc_results['RF (Unscaled)']
rf_change_cv = cv_results['RF (Scaled)'] - cv_results['RF (Unscaled)']
print(f"2. **Random Forest (RF)**: Performance remained stable after normalization")
print(f"    - AUC Change: {rf_change_auc:+.4f}")
print(f"    - CV Acc Change: {rf_change_cv:+.4f}")

# NN Comparison
nn_change_auc = auc_results['NN (Scaled)'] - auc_results['NN (Unscaled)']
nn_change_cv = cv_results['NN (Scaled)'] - cv_results['NN (Unscaled)']
print(f"3. **Neural Network (NN)**: Performance improved significantly after normalization")
print(f"    - AUC Change: {nn_change_auc:+.4f}")
print(f"    - CV Acc Change: {nn_change_cv:+.4f}")

print("\n### Final Conclusion:")
print("Overall, the Random Forest algorithm demonstrated the most robust and high performance across both metrics and normalization states.")
```

--- Overall Model Performance Summary ---

Metric	ROC AUC Score	10-Fold CV Accuracy
LR (Unscaled)	0.845105	0.812562
RF (Unscaled)	0.950843	0.927328
NN (Unscaled)	0.862655	0.839059
LR (Normalized)	0.841954	0.825603
RF (Normalized)	0.950721	0.927245
NN (Normalized)	0.918773	0.887298

--- Comparison and Contrast ---

Best Algorithm Overall (by AUC): **RF (Unscaled)** with **0.9508**

Best Algorithm Overall (by CV Acc): **RF (Unscaled)** with **0.9273**

Impact of Normalization:

- Logistic Regression (LR)**: Performance **improved** after normalization.
 - AUC Change: -0.0032
 - CV Acc Change: +0.0130
- Random Forest (RF)**: Performance remained **stable** after normalization, confirming its relative insensitivity to feature scaling.
 - AUC Change: -0.0001
 - CV Acc Change: -0.0001
- Neural Network (NN)**: Performance **improved significantly** after normalization, as expected, since NN training relies on gradient descent which is highly sensitive to feature scale.
 - AUC Change: +0.0561
 - CV Acc Change: +0.0482

Final Conclusion:

Overall, the **Random Forest** algorithm demonstrated the most robust and highest performance across both unscaled and normalized data. Normalization was crucial for the **Neural Network** and **Logistic Regression** to achieve competitive scores.

In []: